

Système de Détection de Fraude avec Blockchain

Construction pas à pas d'un système complet en Python

Prérequis	Python 3, bases de la POO, CSV
Outils	Python 3.10+, pandas, customtkinter, matplotlib

Introduction et Vue d'ensemble du Projet

Ce TP vous guide dans la création d'un système complet de détection de fraude financière couplé à une chaîne de blocs (Blockchain). Le projet combine plusieurs domaines fondamentaux de l'informatique : la cryptographie, la manipulation de données, et le développement d'interfaces graphiques.

Objectifs pédagogiques

- Comprendre et implémenter les concepts fondamentaux d'une Blockchain
- Créer un pipeline de traitement de données avec pandas
- Développer une règle de détection de fraude basée sur des seuils
- Concevoir deux interfaces graphiques modernes avec customtkinter
- Intégrer des visualisations de données avec matplotlib
- Structurer un projet Python en modules réutilisables

Architecture du projet final

À la fin de ce TP, votre projet aura la structure suivante :

```
fraud-blockchain/
├── data/
│   └── transactions.csv      # Jeu de données des transactions
├── src/
│   ├── __init__.py
│   ├── blockchain.py        # Moteur Blockchain (Block + Blockchain)
│   ├── data_loader.py       # Chargement des données CSV
│   ├── fraud_detector.py    # Règle de détection de fraude
│   └── main.py              # Script console principal
├── gui.py                   # Interface graphique simple
├── gui_advanced.py          # Dashboard complet avec graphiques
└── requirements.txt         # Dépendances du projet
```

INFO : Ce projet est volontairement progressif : chaque étape est indépendante et testable. Lisez chaque section entièrement avant de coder.

Étape 1 — Mise en place de l'environnement

1.1 Installation de Python et de l'environnement virtuel

Avant de commencer, assurez-vous d'avoir Python 3.10 ou supérieur installé sur votre machine.

1. Vérifiez votre version de Python :

```
python --version
# Attendu : Python 3.10.x ou supérieur
```

2. Créez un dossier de projet et placez-vous dedans :

```
mkdir fraud-blockchain
cd fraud-blockchain
```

3. Créez un environnement virtuel et activez-le :

```
# Windows
python -m venv venv
venv\Scripts\activate

# macOS / Linux
python3 -m venv venv
source venv/bin/activate
```

1.2 Création de la structure des dossiers

Créez manuellement (ou via le terminal) l'arborescence des dossiers :

```
# Windows (PowerShell)
New-Item -ItemType Directory -Path src, data
New-Item -ItemType File -Path src/__init__.py

# macOS / Linux
mkdir src data
touch src/__init__.py
```

1.3 Installation des dépendances

Créez le fichier requirements.txt à la racine du projet :

```
pandas
customtkinter
matplotlib
```

Puis installez toutes les dépendances d'un coup :

```
pip install -r requirements.txt
```

CONSEIL : Si customtkinter n'est pas disponible sur votre machine, vous pouvez temporairement utiliser tkinter standard. Toutes les interfaces graphiques fonctionneront avec des adaptations mineures.

Étape 2 — Création du jeu de données

2.1 Comprendre le format CSV utilisé

Le projet repose sur un fichier CSV simple représentant des transactions financières entre utilisateurs. Chaque ligne correspond à une transaction identifiée par quatre champs.

Colonne	Type	Description
id	Entier	Identifiant unique de la transaction
sender	Chaîne	Nom de l'expéditeur
receiver	Chaîne	Nom du destinataire
amount	Entier	Montant de la transaction en dollars

2.2 Créer le fichier data/transactions.csv

Créez le fichier data/transactions.csv et copiez-y le contenu suivant (30 transactions) :

```
id,sender,receiver,amount
1,Alice,Bob,50
2,Charlie,Dave,1500
3,Eve,Frank,200
4,Grace,Hank,5000
5,Ivy,Jack,100
6,Karen,Leo,300
7,Mike,Nina,2500
8,Olivia,Paul,150
9,Quinn,Rachel,800
10,Sam,Tina,1200
11,Uma,Victor,75
12,Wendy,Xander,450
13,Yolanda,Zach,3000
14,Aaron,Bella,250
15,Clara,Derek,950
16,Ethan,Fiona,1800
17,Gina,Hugo,400
18,Ian,Julia,600
19,Kara,Liam,2200
20,Mia,Noah,180
... (continuer jusqu'à la ligne 30)
```

OBJECTIF : Parmi ces 30 transactions, celles dont le montant dépasse 1000 dollars seront considérées comme suspectes. Identifiez-les visuellement avant de coder : ce sont les IDs 2, 4, 7, 10, 13, 16, 19, 22, 25, 28, 30.

Étape 3 — Le module data_loader.py

3.1 Rôle du module

Ce module a une seule responsabilité : lire le fichier CSV et retourner un DataFrame pandas. Il gère également la résolution du chemin de fichier de manière portable (fonctionne depuis n'importe quel répertoire).

3.2 Création du fichier src/data_loader.py

Créez le fichier src/data_loader.py avec le code suivant :

```
import pandas as pd
import os

def load_data():
    # os.path.dirname(__file__) donne le dossier du fichier actuel (src/)
    # '..' remonte d'un niveau pour atteindre la racine du projet
    file_path = os.path.join(os.path.dirname(__file__), '..', 'data',
                              'transactions.csv')
    print(f"Chargement du fichier : {file_path}")
    return pd.read_csv(file_path)
```

3.3 Explication ligne par ligne

Code	Explication
import pandas as pd	Importe la bibliothèque pandas sous l'alias pd
import os	Module standard pour la gestion des chemins
os.path.dirname(__file__)	Retourne le répertoire contenant data_loader.py (= src/)
os.path.join(..., '..', 'data', ...)	Construit un chemin portable : remonte à la racine puis entre dans data/
pd.read_csv(file_path)	Lit le CSV et retourne un DataFrame pandas

3.4 Test du module

Testez ce module seul depuis la racine du projet :

```
python -c "from src.data_loader import load_data; df = load_data();
print(df.head())"

# Résultat attendu :
#   id  sender receiver  amount
# 0   1   Alice      Bob      50
# 1   2  Charlie     Dave    1500
# ...
```

Étape 4 — Le module fraud_detector.py

4.1 La règle de détection

La logique de détection repose sur une règle simple et facilement extensible : toute transaction dont le montant dépasse 1000 dollars est marquée comme suspecte. Dans un système réel, ce seuil serait dynamique et basé sur des modèles statistiques ou de machine learning.

OBJECTIF : Règle : amount > 1000 => Transaction SUSPECTE. Sur nos 30 transactions, 11 dépassent ce seuil.

4.2 Création du fichier src/fraud_detector.py

```
def detect_fraud(df):
    print("Analyse des transactions en cours...")
    # Filtrage par condition booléenne sur la colonne 'amount'
    # df[condition] retourne un DataFrame réduit aux lignes vérifiant la condition
    suspicious = df[df['amount'] > 1000]
    return suspicious
```

4.3 Comment fonctionne le filtrage pandas

L'expression `df['amount'] > 1000` retourne une Série de booléens (True/False pour chaque ligne). En passant cette Série entre crochets, pandas filtre le DataFrame pour ne garder que les lignes True.

```
# Illustration du fonctionnement interne :
masque = df['amount'] > 1000
# masque est une Series : [False, True, False, True, ...]

resultat = df[masque]
# Retourne uniquement les lignes où masque == True
```

4.4 Test du module

```
python -c "
from src.data_loader import load_data
from src.fraud_detector import detect_fraud
df = load_data()
frauds = detect_fraud(df)
print(f'{len(frauds)} transactions suspectes trouvées')
print(frauds[['id', 'sender', 'amount']])
"
```

INFO : Résultat attendu : 11 transactions suspectes, dont les IDs 2, 4, 7, 10, 13, 16, 19, 22, 25, 28, 30.

Étape 5 — Le module blockchain.py

5.1 Concepts fondamentaux de la Blockchain

Une Blockchain est une liste chaînée de blocs où chaque bloc contient des données et une empreinte cryptographique (hash SHA-256) du bloc précédent. Cette propriété garantit l'intégrité de la chaîne : modifier un bloc invalide tous les blocs suivants.

Concept	Définition
Hash SHA-256	Fonction de hachage qui transforme n'importe quelles données en une empreinte de 256 bits (64 caractères hexadécimaux). Déterministe et irréversible.
Bloc Génésis	Premier bloc de la chaîne, dont le champ <code>previous_hash</code> vaut "0" par convention.
Minage	Processus de création d'un nouveau bloc à partir des transactions en attente.
Validation	Vérification que le hash de chaque bloc est cohérent et correspond au <code>previous_hash</code> du bloc suivant.

5.2 La classe Block

Chaque bloc stocke ses données et calcule son propre hash au moment de sa création. Le hash ne doit pas inclure le champ hash lui-même (sinon, calcul circulaire impossible).

```
import hashlib
import datetime
import json

class Block:
    def __init__(self, index, transactions, timestamp, previous_hash):
        self.index = index
        self.transactions = transactions
        self.timestamp = timestamp
        self.previous_hash = previous_hash
        # Le hash est calculé APRÈS avoir assigné tous les autres champs
        self.hash = self.compute_hash()

    def compute_hash(self):
        # CRITIQUE : ne jamais inclure self.hash dans ce dictionnaire
        block_dict = {
            'index': self.index,
            'transactions': self.transactions,
            'timestamp': self.timestamp,
            'previous_hash': self.previous_hash
        }
        # sort_keys=True garantit un ordre constant pour le hachage
        block_string = json.dumps(block_dict, sort_keys=True)
        return hashlib.sha256(block_string.encode()).hexdigest()
```

5.3 La classe Blockchain

```
class Blockchain:
```



```

def __init__(self):
    self.chain = []
    self.pending_transactions = []
    self.create_genesis_block() # Toujours créer le bloc 0

def create_genesis_block(self):
    genesis = Block(0, [], datetime.datetime.now().isoformat(), "0")
    self.chain.append(genesis)

def add_transaction(self, transaction):
    """Ajoute une transaction à la file d'attente"""
    self.pending_transactions.append(transaction)

def mine_block(self):
    """Crée un nouveau bloc avec toutes les transactions en attente"""
    if not self.pending_transactions:
        return None # Rien à miner
    previous_block = self.chain[-1]
    new_block = Block(
        index=len(self.chain),
        transactions=self.pending_transactions,
        timestamp=datetime.datetime.now().isoformat(),
        previous_hash=previous_block.hash
    )
    self.chain.append(new_block)
    self.pending_transactions = [] # Vider la file
    return new_block

def is_valid(self):
    """Vérifie l'intégrité complète de la chaîne"""
    for i in range(1, len(self.chain)):
        current = self.chain[i]
        previous = self.chain[i - 1]
        # Test 1 : le hash stocké correspond-il aux données ?
        if current.hash != current.compute_hash():
            return False
        # Test 2 : le lien avec le bloc précédent est-il intact ?
        if current.previous_hash != previous.hash:
            return False
    return True

```

Étape 6 — Le script principal main.py

6.1 Rôle du script

Le fichier `src/main.py` est le chef d'orchestre du projet en mode console. Il importe les trois modules précédents et les enchaîne dans un pipeline logique en 4 étapes : chargement, détection, enregistrement, validation.

6.2 Code complet de `src/main.py`

```
import sys
import os

# Permet d'importer les autres fichiers du dossier 'src'
sys.path.append(os.path.dirname(os.path.abspath(__file__)))

from data_loader import load_data
from fraud_detector import detect_fraud
from blockchain import Blockchain

def main():
    print("=" * 40)
    print("SYSTEME DE DETECTION DE FRAUDE")
    print("=" * 40)

    # ETAPE 1 : Charger les données
    df = load_data()
    print(f"{len(df)} transactions chargées.")

    # ETAPE 2 : Détecter les fraudes
    frauds = detect_fraud(df)
    print(f"{len(frauds)} transactions suspectes trouvées !")

    # ETAPE 3 : Enregistrer dans la Blockchain
    print("Création de la Blockchain...")
    bc = Blockchain()
    for index, row in frauds.iterrows():
        bc.add_transaction(row.to_dict())
    new_block = bc.mine_block()

    if new_block:
        print(f"Bloc miné avec succès !")
        print(f"Hash du bloc : {new_block.hash[:20]}...")

    # ETAPE 4 : Vérifier l'intégrité
    is_valid = bc.is_valid()
    print(f"Blockchain valide : {'OUI' if is_valid else 'NON'}")
    print("=" * 40)

if __name__ == "__main__":
    main()
```

6.3 Exécution et résultat attendu

```
python src/main.py
```

```
# =====  
# SYSTEME DE DETECTION DE FRAUDE  
# =====  
# 30 transactions chargées.  
# 11 transactions suspectes trouvées !  
# Création de la Blockchain...  
# Bloc miné avec succès !  
# Hash du bloc : a3f8c2e1d9b7f5e4...  
# Blockchain valide : OUI  
# =====
```

Étape 7 — Interface graphique simple (gui.py)

7.1 Présentation de customtkinter

customtkinter est une surcouche moderne à tkinter qui offre des widgets au design soigné avec support du mode sombre. L'application hérite de `ctk.CTk` (équivalent à la fenêtre principale Tk).

7.2 Architecture de la classe FraudDetectionApp

L'interface graphique simple suit le pattern MVC simplifié : la classe centralise à la fois l'état (data, frauds, blockchain) et les callbacks associés à chaque bouton.

```
import customtkinter as ctk
from tkinter import ttk, messagebox
import sys, os

sys.path.append(os.path.join(os.path.dirname(__file__), 'src'))

from src.data_loader import load_data
from src.fraud_detector import detect_fraud
from src.blockchain import Blockchain

ctk.set_appearance_mode("dark") # Mode sombre
ctk.set_default_color_theme("blue") # Thème bleu

class FraudDetectionApp(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title("Système de Detection de Fraude avec Blockchain")
        self.geometry("1200x700")
        self.data = None # DataFrame des transactions
        self.frauds = None # DataFrame des fraudes
        self.blockchain = None
        self.create_widgets()
```

7.3 Ordre de création des widgets

La méthode `create_widgets()` construit l'interface dans cet ordre précis :

4. Titre principal centré en haut
5. Barre de boutons horizontale (Charger / Détecter / Blockchain)
6. Panneaux de statistiques (4 cartes colorées)
7. Tableau de données (ttk.Treeview avec scrollbar)
8. Section d'informations Blockchain
9. Barre de statut en bas

7.4 Gestion des états des boutons

Un point important du design : les boutons Détecter et Créer Blockchain sont initialement désactivés (`state='disabled'`). Ils ne s'activent qu'après l'étape précédente réussie, ce qui guide

l'utilisateur dans le bon ordre d'utilisation.

```
# Activation progressive des boutons :  
# Étape 1 : Charger les données => active le bouton 'Détecter'  
self.btn_detect.configure(state="normal")  
  
# Étape 2 : Détecter les fraudes => active le bouton 'Créer Blockchain'  
self.btn_blockchain.configure(state="normal")
```

7.5 Le tableau de données avec ttk.Treeview

Le widget ttk.Treeview est utilisé pour afficher les transactions sous forme de tableau. Le style est personnalisé via ttk.Style pour s'harmoniser avec le thème sombre.

```
style = ttk.Style()  
style.theme_use("clam")  
style.configure("Treeview",  
    background="#0f3460",    # Fond des cellules  
    foreground="white",      # Couleur du texte  
    fieldbackground="#0f3460",  
    rowheight=30)  
  
self.tree = ttk.Treeview(  
    table_frame,  
    columns=("id", "sender", "receiver", "amount", "status"),  
    show="headings"          # Cache la colonne d'index interne  
)
```

7.6 Exécution de l'interface simple

```
python gui.py
```

INFO : Flux d'utilisation : (1) Cliquez 'CHARGER LES DONNEES' -> tableau rempli. (2) Cliquez 'DETECTER LES FRAUDES' -> les suspects apparaissent en rouge. (3) Cliquez 'CREER BLOCKCHAIN' -> informations du bloc affichées.

Étape 8 — Dashboard avancé avec graphiques (gui_advanced.py)

8.1 Différences avec l'interface simple

Le fichier `gui_advanced.py` propose un tableau de bord complet avec une navigation par onglets, des graphiques matplotlib intégrés, et une option d'export des données.

gui.py (simple)	gui_advanced.py (dashboard)
Interface à écran unique	Navigation sidebar multi-vues
Pas de graphiques	4 graphiques matplotlib intégrés
Statistiques basiques	Statistiques avancées (moy, max, min)
Pas d'export	Export CSV et Excel via <code>filedialog</code>

8.2 Architecture du Dashboard

La classe `FraudDetectionDashboard` organise l'interface en trois zones :

- Header : barre de titre fixe en haut
- Sidebar : menu de navigation latéral à gauche (200px de large)
- `content_frame` : zone centrale dynamique qui change selon la vue active

8.3 Système de navigation par vues

Chaque bouton de la sidebar appelle une méthode `show_XXX_view()` qui efface d'abord tous les widgets de `content_frame`, puis reconstruit la vue correspondante.

```
def show_dashboard_view(self):
    # Effacer l'ancienne vue
    for widget in self.content_frame.winfo_children():
        widget.destroy()
    # Construire la nouvelle vue
    # ...

# Les 4 vues disponibles :
# show_dashboard_view()    => Tableau de bord + stats + actions
# show_data_view()         => Tableau complet des transactions
# show_analytics_view()    => Graphiques matplotlib
# show_blockchain_view()   => Détails de la blockchain
```

8.4 Intégration de matplotlib

matplotlib est intégré à tkinter via `FigureCanvasTkAgg`, qui transforme une figure matplotlib en widget tkinter affichable dans n'importe quel `Frame`.

```

import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

def show_analytics_view(self):
    # Créer une figure avec 4 sous-graphiques (2x2)
    fig, axes = plt.subplots(2, 2, figsize=(10, 8))
    fig.patch.set_facecolor('#161b3d') # Fond sombre

    # Graphique 1 : Histogramme des montants
    axes[0, 0].hist(self.data['amount'], bins=10, color='#00d9ff')

    # Graphique 2 : Top 10 transactions (barres horizontales)
    top_10 = self.data.nlargest(10, 'amount')
    axes[0, 1].barh(range(len(top_10)), top_10['amount'], color='#e94560')

    # Graphique 3 : Camembert statistiques
    axes[1, 0].pie([...], labels=[...], autopct='%1.1f%%')

    # Graphique 4 : Scatter plot avec seuil de fraude
    axes[1, 1].scatter(self.data['id'], self.data['amount'])
    axes[1, 1].axhline(y=1000, color='#e94560', linestyle='--')

    # Intégrer dans tkinter
    canvas = FigureCanvasTkAgg(fig, master=canvas_frame)
    canvas.draw()
    canvas.get_tk_widget().pack(fill='both', expand=True)

```

8.5 Exécution du dashboard

```
python gui_advanced.py
```

Étape 9 — Exercices d'extension

OBJECTIF : Ces exercices sont facultatifs mais fortement conseillés pour approfondir votre compréhension du projet.

Exercice 1 — Seuil paramétrable

Modifiez `fraud_detector.py` pour que le seuil de détection (actuellement 1000) soit passé en paramètre de la fonction `detect_fraud()`.

```
# Signature actuelle :
def detect_fraud(df):

# Signature demandée :
def detect_fraud(df, threshold=1000):
    # threshold est maintenant configurable par l'appelant
    suspicious = df[df['amount'] > threshold]
    return suspicious
```

Mettez à jour `main.py` et les deux interfaces graphiques pour permettre à l'utilisateur de saisir ce seuil.

Exercice 2 — Preuve de travail (Proof of Work)

Une vraie Blockchain exige un travail computationnel pour miner un bloc. Ajoutez un mécanisme de Proof of Work dans la classe `Blockchain` : le hash doit commencer par un certain nombre de zéros.

```
def mine_block_pow(self, difficulty=2):
    prefix = '0' * difficulty # Ex: '00' pour difficulty=2
    # ...
    while not new_block.hash.startswith(prefix):
        new_block.nonce += 1 # Ajouter un champ nonce à Block
        new_block.hash = new_block.compute_hash()
    return new_block
```

Exercice 3 — Détection multi-critères

Enrichissez `detect_fraud()` avec deux règles supplémentaires de fraude et retournez les transactions suspectes avec une colonne indiquant la raison.

- Règle 1 (existante) : montant > 1000
- Règle 2 (nouvelle) : même expéditeur apparaît plus de 3 fois dans le CSV
- Règle 3 (nouvelle) : montant rond (multiple de 1000)

Exercice 4 — Persistance de la Blockchain

Ajoutez deux méthodes à la classe Blockchain pour sauvegarder et charger l'état de la chaîne depuis un fichier JSON.

```
def save_to_file(self, filepath):  
    # Sérialiser self.chain en JSON  
    pass  
  
def load_from_file(self, filepath):  
    # Désérialiser et reconstruire self.chain  
    pass
```

Récapitulatif et grille d'évaluation

Checklist de livraison

- Structure de dossiers conforme à l'arborescence du projet
- Fichier data/transactions.csv avec 30 lignes
- Module src/data_loader.py fonctionnel et testé
- Module src/fraud_detector.py fonctionnel et testé
- Module src/blockchain.py avec Block et Blockchain
- Script src/main.py exécutable depuis la racine
- Interface graphique gui.py avec les 3 boutons et le tableau
- Dashboard avancé gui_advanced.py avec les 4 vues
- Au moins un exercice d'extension réalisé

Grille d'évaluation

Critère	Points	Barème
Structure du projet et organisation des fichiers	/4	Arborescence, init, imports
Module data_loader.py (chemin portable, lecture CSV)	/3	os.path, lecture correcte
Module fraud_detector.py (filtrage pandas)	/3	Condition booléenne, résultat correct
Module blockchain.py (Block + Blockchain + is_valid)	/5	SHA-256, chaînage, validation
Script main.py (pipeline complet)	/3	Enchaînement des 4 étapes
Interface gui.py (widgets, états, tableau)	/5	UX, gestion états boutons
Dashboard gui_advanced.py (navigation, graphiques)	/5	4 vues, matplotlib intégré
Exercice d'extension (au choix)	/2	Fonctionnel + commenté
TOTAL	/30	

CONSEIL : Rendu : un dossier compressé (.zip) contenant l'intégralité du projet. Le nom du fichier doit être : NOM_Prenom_TP_FraudBlockchain.zip